PinDrop: Breaking the Silence on SDCs in a Large-Scale Fleet

Peter W. Deutsch*, Harish D. Dixit†, Gautham Vunnam†, Carl Moran†, Eleanor Ozer†, and Sriram Sankar†
* MIT/Meta, Cambridge, Massachusetts, pwd@mit.edu
†Meta, Menlo Park, California, {hdd | gauthamvunnam | carl.moran1 | ozer | sriramsankar}@meta.com

2026 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-9302-5/26/$31.00 ©2026 IEEE | DOI: 10.1109/HPCA68181.2026.11408620

***Abstract*—Silent Data Corruptions (SDCs) pose a significant and often hidden threat to the reliability of large-scale computing infrastructure, as they can silently compromise data integrity without immediate detection. Detecting such behaviors at hyperscale is often challenging due to their intermittent nature and the vast diversity of hardware and workloads in a large-scale fleet. This work addresses these challenges by introducing PinDrop, a characterization methodology that leverages continuous, high-frequency testing infrastructure across millions of servers to gather information about SDCs at scale. By leveraging extensive test-suites tailored to mimic real-world applications and exercise a wide range of CPU features, we provide the most comprehensive characterization of SDC failures to date, analyzing over 500 million test executions across millions of devices. Our findings reveal that 0.035% of tested machines suffer from at least one SDC failure during their lifetime. Examining years of data (rather than just a testing snapshot in time), we observe SDCs emerging long after initial deployment and persisting over time. Detailed analysis shows that an average of 0.0024% of tested machines begin failing in each quarter they are tested beyond an initial burn-in period, confirming a fundamental need for continuous testing. Our findings also provide further insights into SDC behaviors at-scale, including failure breakdowns across architectures, test families, specific core IDs, and output-level behaviors.**

I. Introduction

In recent years, a steady stream of reports from hyperscalars including Google [12], [23], Meta [7], and Alibaba [36] have indicated that significant numbers of processors in their fleets are experiencing Silent Data Corruptions (SDCs). SDCs represent cases where a processing unit silently provides an incorrect result (e.g., $1+1=3$) without detecting that an error occurred, leading to downstream instructions and services using the incorrect result. SDCs represent an insidious threat to computing, leading to production issues requiring significant engineering resources to identify and debug. Several previous SDC characterization works have identified a wide variety of affected workloads, including silent deletions in storage/-database systems [2], [7], [8] and interruptions during ML training [11], [15], [20].

The scale of the SDC challenge has grown substantially in recent years, driven by both the increasing internal complexity of modern processors (both at the architectural and circuit-levels) and the massive deployment of compute at hyperscalars today. While originally considered an exceedingly rare phenomenon, recent industry publications [7], [12], [23], [36] have suggested that one in every few thousand machines can be observed to exhibit SDC behaviors. Unfortunately, failures at this rate make SDCs paramount to address but difficult to characterize – SDCs are common enough to warrant immediate concern, but rare enough that millions of machines are required in order to understand the overall behavior of failures at scale.

Recent papers from hyperscalars have offered insights into real SDC behaviors observed in their fleets. Initial papers on SDCs from hyperscalars [12], [22] highlighted anecdotes about SDC behaviors, discussing common symptoms and failure patterns that were observed in production workloads. To better characterize the scope of the SDC issue, more recent works [7], [23], [27], [36] have explored dedicated testing strategies to detect machines suffering from SDCs and remove them from production. Through applying directed and randomized tests, these initial efforts have been able to root out in the order of hundreds of faulty machines which fail across a wide variety of features. These prior works have faced challenges in fully characterizing SDC behaviors at scale, however. Due to the rarity and stochastic nature of SDC failures, millions of machines must be tested over extended periods to extract meaningfully rich data about failure patterns. To address this challenge, the largest SDC study to date [36] relied on a snapshot-in-time testing approach, performing a one-time testing sweep by sequentially testing machines for SDCs in batches. While this approach enabled the study of a large machine population, we note that it can miss intermittent or late-onset failures that only manifest after prolonged use or after extensive testing (such as wearout, marginal defects, or input/seed-dependent behavior).

This Paper. In this work, we present PinDrop: a characterization framework enabling the evaluation of SDC test behaviors within Meta's large-scale computing fleet, spanning millions of machines. Via PinDrop, we have gathered data from over 500 million SDC tests on a machine population deployed over 12 years, leading to the detection of thousands of suspected faulty machines. SDC observations obtained from PinDrop bolster prior industry reports on machine failure rates, revealing that 0.035% of tested machines at Meta suffer from at least one SDC failure during their lifetime. For the first time, we provide data indicating that wide varieties of failure behaviors can emerge even after years of testing, highlighting the fundamental need for continuous testing rather than snapshot-in-time approaches. We also characterize failure persistence,

showing that over 70% of failing machines at Meta continue to fail consistently over multiple years of sustained testing. Our analysis further reveals detailed failure breakdowns based on test type and CPU architecture, revealing further insights into core-level failure affinities and other at-scale failure behaviors.

Key Contributions. The key contributions of this work are:

1) PinDrop: A characterization framework leveraging data gathered from a continuous, high-frequency SDC testing infrastructure employed at Meta. PinDrop enables the characterization of long-term SDC behaviors, overcoming the limitations of "snapshot-in-time" based testing approaches.
2) A comprehensive review of SDC test failures at fleet scale using PinDrop, studying millions of production machines and over 40 million test executions per month.
3) The first examination of long-term SDC failure trends over an extended four-year period. We identify new insights into SDC failure persistence and how long it takes for a machine to begin to fail – including identifying cases where new SDC failures were identified only after years of testing.
4) Detailed SDC failure characterizations studying a diverse set of tested workload types; CPU architectures, generations, and core IDs; and faulty instruction outputs.

Paper Overview. We begin in Section II by describing the impact of silent data corruptions at scale, their potential hardware root-causes, and a summary of existing public efforts to detect failures in large fleets. In Section III, we then describe Meta's testing infrastructure and machine population – describing how tests are scheduled, data is collected, and ultimately processed by PinDrop. In Section IV, we will highlight the different types of SDC tests run across the studied fleet. The presentation of SDC failure result characterizations begins in Section V, first highlighting high-level failure characteristics and failure trends over time. In Section VI we perform a deep-dive into observed SDC failure behaviors, including the analysis of broad-scoped tests in Section VI-A and vector-based tests in Section VI-B.

II. Background & Motivation

A. Impact of SDCs at Scale

SDCs have a wide range of impact in large-scale applications. Meta has observed application-level impacts due to silent data corruptions across recommendation systems, caches, databases, compression, querying infrastructure, storage and backup systems, AI training, and inference in the machine population under study.

SDC-related failures have manifested in a variety of ways across these applications at Meta. Computational errors in arithmetic instructions have resulted in miscomputations in pipelines associated with inference applications, including recommendation systems. Corruptions in concurrency related features have resulted in resource livelocks due to starvations across applications. Data movement corruptions have resulted in data saved, queried, and retrieved from databases to be corrupt in different stages of the save and retrieval process, such as compression, decompression, collectives, direct memory accesses, and query outputs. Corruptions in storage applications have caused latent data corruptions, where a computational unit generating a checksum wrote an erroneous checksum across all replicas, and subsequent data retrieval after many months resulted in failures. AI training applications have even broader challenges, with propagation of erroneous values rendering training runs useless or resulting in fault contagions requiring complex triage across large clusters.

Each of the aforementioned scenarios was caused by a corruption within a single CPU serving a distributed application, ultimately affecting an entire service. All of these manifestations have resulted in meaningful application impacts. As there is no residue within the system of said corruption, the triage process typically involves hundreds of engineers instrumenting at every layer of the stack to detect variances. Typical debug for a single application-level data corruption can last from a few weeks to months. After investigations, failure reproduction, sample evaluation, and reproducer test construction, these reproducer tests are deployed at fleet scale (as will be discussed in Section III). Test-suites have been effective in detecting corruptions proactively and in reducing application-level impact, however we note that test escapes still occur across all computational units. Further, the novelty of the challenge still persists when considering non-CPU-based accelerators, where testing capabilities are nascent.

B. Potential Root-Causes of SDCs & Testing Implications

Due to a lack of visibility into the underlying hardware, the root-causes of SDCs are often not well understood. A large number of emergent SDC behaviors recently reported by hyperscalers cannot be attributed to particle-strike induced transient faults [6], [7], [23], [36]. Instead, recent reports from chip vendors suggest one potential common cause of SDCs may be marginal defects [5], [21], [31]. These marginal defects can result in small delay faults (or other erroneous circuit behaviors) that only occur under specific temperature, frequency, voltage, program, and/or input combinations. This sensitivity makes it difficult for chip vendors to develop exhaustive quality control tests to identify defects, leading to test escapes [23]. Often, SDCs are first observed in real workloads, with internally developed test reproducers and debug information then being shared with chip vendors for further in-depth study. Chip vendors then use this information to develop both better chip-level mitigations and quality-assurance tests for future architectural revisions.

C. Prior SDC Test Characterization Works

The most comprehensive test-based characterization of SDCs in a large-scale fleet was published by Alibaba in 2023 [36]. In their characterization, more than a million machines were tested, identifying SDC test failure trends across hundreds of faulty CPUs. While being the first to provide a comprehensive overview of SDCs observed in a

production environment, the Alibaba characterization leaves several key open questions due to how testing was performed. Claiming that *it is not feasible to perform regular SDC tests frequently*, their characterization relied on testing machines in groups, with each group being tested for two weeks, consequentially taking several months to test their way through the production fleet. This testing strategy effectively provided an SDC *snapshot-in-time* – machines were only subjected to testing for a few weeks, after which machines which did not fail during those weeks were deemed non-faulty. We note however that there are many cases where an ultimately faulty machine may not immediately fail SDC tests – machines may only start observing failures later in their life (e.g., due to wearout [12]), suffer from intermittent failures (e.g., due to marginal defects [32]), or may simply require a large number of test runs (which are often stochastic in nature) to uncover the problem. These cases lead to blind spots when employing snapshot testing approaches, potentially undercounting the number of machines that fail over time.

Given these limitations, infrastructure that allows for *continuous* testing of machines over the course of their lifetime is needed to better illuminate failures at-scale. In contrast to snapshot-in-time approaches, Meta employs continuous in-production testing methodologies at-scale, as described in the Fleetscanner/Ripple [6] work, however limited details have been presented so far on failure categorization of tests, testing cadences, test efficacy, and architectural behaviors of failures. In this paper, we provide an exhaustive view of this SDC detection methodology implemented at Meta across a significantly larger observational window. Meta has also previously reported on Hardware Sentinel [9] which approaches detection purely analytically via examining data and control plane exceptions. While Hardware Sentinel showcases that analytical methods can be beneficial in detecting SDCs without dedicated tests, Meta has detected many SDC behaviors through applying both approaches.

III. Testing Infrastructure and Machine Population

In this section we'll describe how test data was gathered and processed, providing key statistics on how frequently and thoroughly machines were tested. The fleet studied by PinDrop consists of millions of servers across numerous server configurations and architectures located in different data centers. The machine population under study consists of devices deployed in the fleet from 2014 to 2025, spread across all available server configurations and architectures. The particular data presented in this paper has been aggregated from tests performed over the past 4 years.

A. Infrastructure Reservation Management System (IRMS)

In Meta's large-scale infrastructure, numerous applications performing front-end web, cache/database, storage/backup, and AI tasks are executed across the fleet. The scheduling of these workloads is implemented using an infrastructure reservation management system (IRMS) similar to systems described in [18], [34]. Based on the type of workload, expectations of service level agreements (SLAs), and the availability of hardware, applications are scheduled on appropriate infrastructure configurations. An infrastructure configuration is defined as the unique set captured by the server configuration, CPU architecture, data center and region, kernel version, firmware and BIOS versions, and unique workload. At any point in time within our large-scale infrastructure, a few hundred thousand of such configurations co-exist. At any given time, the IRMS catalogs all of the servers in the hyperscale infrastructure. Most will be marked as available/healthy (and ready to accept jobs), while a subset of servers will be undergoing maintenance, provisioning, or repairs, and will be marked as unavailable or unhealthy. Further details about the system are described in [35]. Among the healthy/available servers, reservations are placed to allocate workloads. These reservations can last for smaller durations such as hours to longer durations ranging from a few months to a year.

To facilitate maintenance operations and to also enable frequent testing of hardware within the fleet, policies are set where services and servers are regularly marked for maintenance or updates, enabling test reservations to preempt existing jobs on these machines. The testing priority is configurable and varies differently for different infrastructure configurations. The hyperscale fleet maintains different prioritization levels such as P0, P1, P2, P3, P4, etc., with P0 representing highest priority and the fastest cadence used for deployments and testing. Broadly, the test reservation policy can be defined as specifying the desired testing cadence for the entire fleet on average. The testing phase within IRMS represents an advanced implementation of the methodology in *Fleetscanner*, captured in [6]. At a server level, this translates to each server being marked for a testing reservation at a specific frequency as represented by *N servers every X days* in Figure 1.

Servers marked for test reservations have their workloads evicted and tests scheduled. Upon test completion, servers that pass the tests are sent back to service workload reservations. Machines that fail tests are placed in a longer reservation hold for deeper investigation. The longer reservation hold allows for continuous execution of tests and enables the optimization of tests for faster detection for similar failure signatures. After the expiration of the hold (which in some cases can last for years based on capacity allocation policies) the servers are released back to production work after replacement of the faulty part.

B. PinDrop: A Characterization Framework

We now present PinDrop, a characterization framework used to characterize the results of all the components of IRMS at scale for SDCs. PinDrop primarily consists of three components – 1) the Partitioning Engine, 2) Characterization Engine, and 3) Querying Infrastructure. Via PinDrop, we aggregate architectural and workload information from all the servers within the infrastructure, including transitions related to operating state, kernel versions, firmware upgrades, workload allocations, and evictions. For each of the servers under test reservations, test parameter aspects associated with test

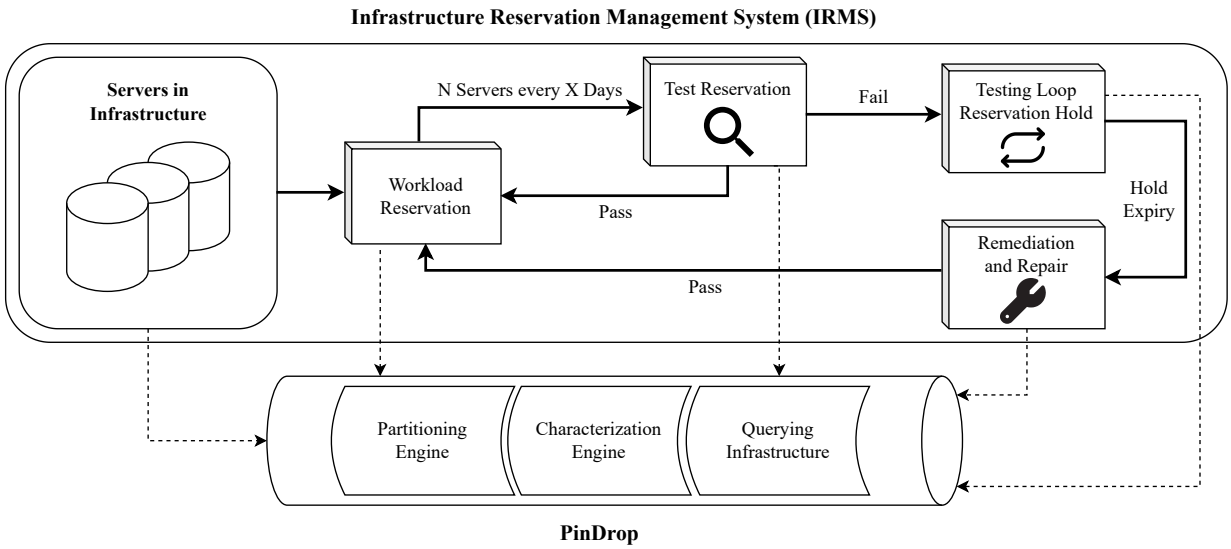

Fig. 1. A summary of the PinDrop characterization framework alongside the Infrastructure Reservation Management System (IRMS). Dotted lines represent data flow to PinDrop.

Fig. 2. Average tests executed per month by IRMS at scale over time.

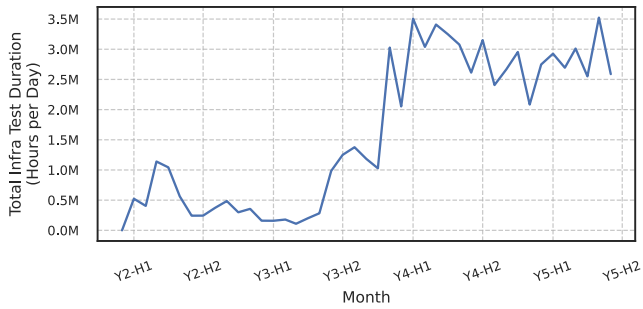

Fig. 3. Daily total of test duration in infrastructure over time.

execution, log features associated with test results, residency in testing reservation states, and remediation transitions are captured.

In the Partitioning Engine, data from test workloads and infrastructure is captured on a daily basis. Resultant data is aggregated and stored on a daily basis with a retention expectation of 6 years. As of July 2025, over 500 million SDC test executions have been recorded by the Partitioning Engine. After deduplication, and joining across unique tests

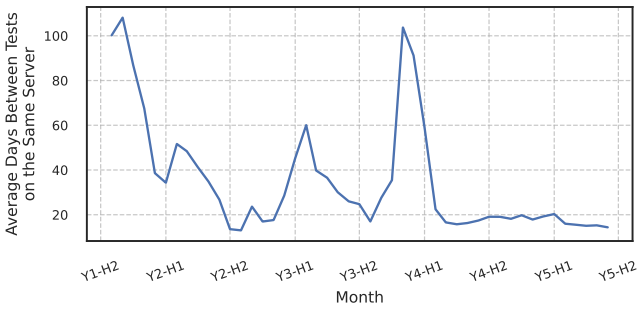

Fig. 4. Average days between tests on the same machine over time.

and infrastructure configurations, greater than 10M entries are registered into the database with an average compressed daily footprint of 8 GB. Overall, the PinDrop SDC test result database under study has a footprint of 8 terabytes.

The Characterization Engine consists of all key data splicing operations, performing correlations across fields like CPU architecture, test runtimes, time between tests, core counts, test outputs, subtest features, workloads, time to failure, aggregate of number of bitflips, total tests executed, asset transitions, etc. (as will be dissected in Section V). In total, more than 110 features are spliced across 2- or 3-way correlations. The engine maintains a relational database across a number of these features.

The Querying infrastructure enables on-demand accessibility to the data using the Presto [28] querying framework. This enables for fast retrieval of data from the Characterization Engine's outputs, along with an option to conduct characterizations outside of the pre-existing splices.

C. Infrastructure Observations

Using IRMS and observing the testing policies associated with the hyperscale fleet, we now present numerous infras-

tructure artifacts related to testing at scale. In this section, we capture a few key aspects about the reservation system to showcase the periodic and frequent nature of testing required to detect SDCs at scale.

Dates described in figures and artifacts in this section are labeled using the convention *Y1-H2*, indicating the second half of the first year in which testing began.

Infrastructure Artifact 1. PinDrop characterizes results based on the execution of 40 million test executions per month while allocating 3 million hours of daily testing reservations over a 50-month period.

The reservations for tests associated with silent data corruptions were bumped to a higher priority in *Y3-H1*, from P4 to P3 (representing a target cadence of less than 60 days), and subsequently to an even higher priority, from P3 to P2 (representing a target cadence of less than 30 days) in *Y3-H2* due to the increasing occurrences of data corruptions at scale. Target cadences are revisited across infrastructure periodically for all services. We recognize that there may be implications to our results due to these priority changes in reservation allocation.

The system achieved approximately 10 million test executions per month in *Y1-H2* and has scaled up to 40 million test executions per month at the time of publication (as shown in Figure 2). We capture the trend associated with the average testing duration across infrastructure from *Y2-H1* to *Y5-H2*, since instrumentation associated with duration was enabled in later stages of *Y1-H2*. As of July 2025, across all the visits to servers, approximately 3 million hours per day on average are reserved for silent data corruption focused tests, as reflected by the trend in Figure 3.

While the volume of testing reservations is significant, in order to ensure coverage across the breadth of Meta's fleet, the cycling cadence across servers is a key metric to evaluate. We capture this cycling cadence by measuring the average days between SDC tests on the same server in Figure 4, representing how frequently we visit a particular server. When the prioritization was switched, from P4 to P3, and from P3 to P2, we notice a corresponding spike in the days between visits. This is a result of the increased priority – while a broader initial reach is possible due to the priority change, this initial reach is achieved at the cost of frequency. These spikes eventually settle down to a lower average number of days as represented in Figure 4, showcasing long-term improvements to coverage.

We observe from the trends in Figure 5 that IRMS visits more than 25,000 unique configurations of server model, kernel, datacenter, workload and firmware versions any given day. This corresponds to a daily reach of 15% of all unique available configurations in hyperscale infrastructure, representing a minimum reach of approximately 1.75% of all available testable infrastructure at Meta every day.

Fig. 5. Number of unique configurations reached daily across the infrastructure over time

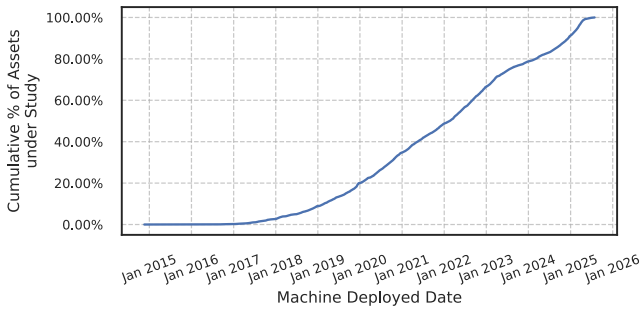

Fig. 6. Cumulative percentage of machines under study versus machine deployed date.

Infrastructure Artifact 2. PinDrop characterizations are based on a revisitation cadence that can be as short as 15 days on average for the same server, while covering more than 1.75% of all available infrastructure every single day.

In order to quantify the coverage and heterogeneity of the data corruption characterization study, we capture the deployment dates of all servers reached by IRMS. The results in this study showcase observations made on servers deployed between 2014 to 2025. Figure 6 captures the cumulative asset percentages tested versus the machine deployment date, indicating the breadth associated with our periodic testing based observations. A characterization study of this heterogeneity, scale, and span of time is unique in comparison to any published literature on silent data corruption.

Infrastructure Artifact 3. PinDrop characterizes results across 200,000+ unique configurations, 200+ server configurations, 500+ bios versions, and 100+ workload families from infrastructure deployed between 2014 to 2025.

D. Limitations and Nuances

The efficacy of IRMS is directly dependent on infrastructure policies related to reservation, as each increase in priority for the reservation substantially improved the testing allocation. However, it is important to note that these increases in testing reservations are fundamentally undesired for a large-scale

infrastructure. The goal is to keep the reservation allocation as low as possible while increasing the efficacy of tests. Ideally, the infrastructure preference is to have no allocation for testing hardware faults at scale, however the reality is that faults are frequent and significant in their impact, necessitating these surveys. While the average revisit per host is 15 days, there are outliers which may take longer to obtain a reservation allocation (e.g., due to datacenter maintenance), and the reservations mostly represent a distribution, as commonly observed in large-scale distributed systems.

In this paper, our intent is to specifically focus on gathering and presenting data associated with architectural aspects of machines under test. As such, we do not analyze any potential associations of SDCs with variations in datacenter halls, kernel versions, BIOS versions, or other IRMS variability factors.

IV. Tests Performed

Meta's hyperscale fleet is tested using a blend of tests which are uniquely configured for test runtime, instruction coverage, similarities to application profiles, and the capability to reproduce failures across stress, power, and performance profiles. Our library of over a thousand tests consists of a blend of tests received from vendors and tests developed in-house. This library of tests has grown across architectural generations to test new features and learnings from failures in the field. These tests are written using different layers of the stack; starting from assembly based tests to application representative tests. During a test reservation, a random selection of tests from a particular *test-suite* (i.e., a pre-defined group of tests) are executed while respecting the reservation priorities. The cumulative runtime of a single test reservation is on the order of minutes. A single test reservation is often too short to run all tests in a test-suite, however machines are regularly tested many times across different test-suites and thus good test coverage across a test-suite is achieved in aggregate.

Test Structure. Each test aims to exercise a particular architectural functionality that has previously been observed to fail when running production workloads (as described in Section II-B). Each test exercises all cores simultaneously to check for failures. Depending on the test, failures are detected via either 1) a deviation from an expected ("golden") output, or 2) a deviation between the results generated across cores on the machine. A majority of tests include some amount of non-deterministic behavior – for non-deterministic tests a new random seed is chosen for each test execution. Upon any observed failure, the test seed associated with the failure is recorded for future diagnostic purposes.

Test Families. In practice, Meta's large hyperscale fleet hosts a variety of application profiles. These are broadly classified as representing front-end web applications, virtual machine instances, encryption and decryption applications for security, encoding and decoding applications, compression and decompression applications, caching services and databases, storage, and AI applications (among others). Production failures observed in each of these application profiles are studied, and tests are developed to exercise the instructions and architectural behaviors associated with those failures. Across all test applications considered (across all suites), basic arithmetic and instruction control and branching operations are expected.

In this paper, we categorize our tests across 9 broad test families, encapsulating the different behaviors of applications that are utilized at infrastructure scale.

The 9 test families are:

1) **Arithmetic**: Standard arithmetic and math operations.
2) **Concurrency**: Lock-based concurrency semantics.
3) **Control Path**: Instruction sequences mimicking control and conditional code paths.
4) **Data Move**: Large-scale data transfer and replication.
5) **Floating Point Unit**: Floating-point arithmetic.
6) **Special**: Encoding, decoding, encryption, and decryption.
7) **Transaction**: Transactional memory synchronization.
8) **Vector**: Large-scale matrix multiplications, vector operations, and AI applications.
9) **Other**: The aggregated bucket of other applications.

In the following sections, we will share findings associated with this broad test characterization. We examine tests over several test-suites, one of which being a *broad-scoped* test-suite (covering all test families), and others focusing on specific functionalities (e.g., vector functions). Tests from these suites are executed in an interspersed manner amidst test reservations. For some tests in the broad-scoped test-suite, some tests that exercise multiple functionalities can simultaneously fall into more than one test family. All machines, at any given point of time, draw from the same set of tests across the entire fleet. Changes to testing, such as modifying test duration and the composition of tests, are consistently evaluated and updated.

V. SDC Test Characterizations in a Large-Scale Fleet

Through the remainder of this paper, we'll describe the key insights gained through PinDrop on Meta's large-scale CPU fleet. In Section V-A, we'll begin by highlighting general observations made across all 1000+ tests run in our fleet, providing high-level insights into machine vulnerability across 8 examined CPU architectures. In Section V-B, we'll analyze time-series failure data gathered by PinDrop, examining how failure behaviors develop and sustain themselves over time in the fleet. In Section VI, we'll then zoom in and perform a deep-dive into specific failure observations made by PinDrop. In particular, in Section VI-A we will examine failure patterns observed across a subsection of broad-profile tests, then zoom in on vector-related tests in Section VI-B.

A. High-Level Failure Characteristics

A breakdown of the proportion of all machines that were observed to experience at least one SDC test failure, classified across different (anonymized) CPU architectures, is shown

TABLE I
PERCENT PROPORTION OF MACHINES THAT EXPERIENCE AT LEAST ONE SDC TEST FAILURE ACROSS ALL TEST-SUITES, SPLIT BY ARCHITECTURE.

CPU Architecture	A	B	C	D	E	F	G	H
% of Failing Machines	0.037009	0.021836	0.014364	0.037124	0.178081	0.038490	0.024348	0.022974

in Table I. We note that we have executed a significantly richer set of tests on CPUs of architectures A-F (spanning across all categories listed in Section IV) compared to those of architectures G-H (which are limited to tests exercising vector and generic stress-tests).

Observation 1. Overall, 0.035% of all machines (roughly 1 in 2,850) tested are observed to fail at least once across all tests performed.

Our observation of failures in one in every few thousand machines is in line with claims previously made by Meta [7], Google [12], [23], and Alibaba [36]. Architecture E had the highest failure rate, with 0.178% of tested machines observed to experience at least one SDC test failure. We note that this architecture is older, predates significant improvements to quality assurance (both in-house and by the vendor), and exhibited correspondingly high failure rates in production.

Due to the previously mentioned limited diversity of tests run on architectures G-H, we will restrict our analysis to architectures A-F (unless otherwise specified) for the remainder of the paper. Furthermore, as different architectures may be subjected to varying overlapping sets of tests (e.g., new tests can be introduced for new architectural features), we do not make any conclusions about whether one architecture is more vulnerable than another.

We further break down the proportion of all tested machines that fail at least once in each test family (as defined in Section IV) in Figure 7.

Observation 2. Failures are not limited to basic computational/arithmetic functions, and are regularly experienced across a wide range of complex functionalities (including encryption, transactional memory, etc.).

While arithmetic failures have been the primary focus of several well-known case studies in literature [7], [12] and tend to be some of the most vulnerable functionalities by observation rate, our results suggest that a wide range of complex architectural features (including encryption, AI-enabling matrix instructions, among others) are also vulnerable. This vast array of feature failures indicates a continued need to deploy complex tests at scale in order to capture faulty machines in practice.

We also analyze the proportion of failing machines that experience failures across one, two, three, or more than three different test families, shown in Figure 8.

Observation 3. Across all architectures, a majority of failing machines only experience failures associated with one test family, however instances where machines experience failures across multiple test families do regularly occur.

B. Failures Trends Over Time

Drawing from over 4 years of data, PinDrop has been used to track the failure rates of SDC tests over time. The likelihood that a machine experiences a failure, and the rate at which a failing machine continues to fail, can potentially vary over the course of the machine's lifetime. For instance, younger machines may suffer from infant mortality (not detected by the vendor/initial acceptance testing), and older machines may suffer from aging and wear-out effects. While the characterization provided by PinDrop cannot determine *why* a machine may have failed an SDC test at a given time, we now use PinDrop to examine whether we observe hints of these age-related effects in our machine population over time.

First Failure Time. We first examine how long it takes for a machine to experience its first SDC test failure in the fleet. Figure 9a) denotes when tested machines experienced their first failure, measuring from the time elapsed since the test-suite was first run on the machine. In cases where a test-suite was updated partway through the machine's lifespan (i.e., introducing new tests), we ignore failures from that test-suite for the next 30 days for the purposes of these time-series studies. While such failures are still indicative of a machine failure (and considered in other analyses), we exclude failures in such a 30-day window in this section as it is unclear when that failure would have been observed if the newly introduced test was introduced earlier.

Observation 4. Machines can begin to experience failures even after years of testing, indicating a strong need for continuous testing.

While machines often fail immediately after we begin testing, we observe a considerable number of them fail after continued testing occurs. Notably, we observe first failures on machines that have been tested in production for almost four years, with one machine first tested in August 2021 failing its first test in July 2025. In general, we see continued new failures at similar rates over time across most test categories, with two outliers being transactional and concurrency-related tests which are typically discovered in the first two quarters of testing.

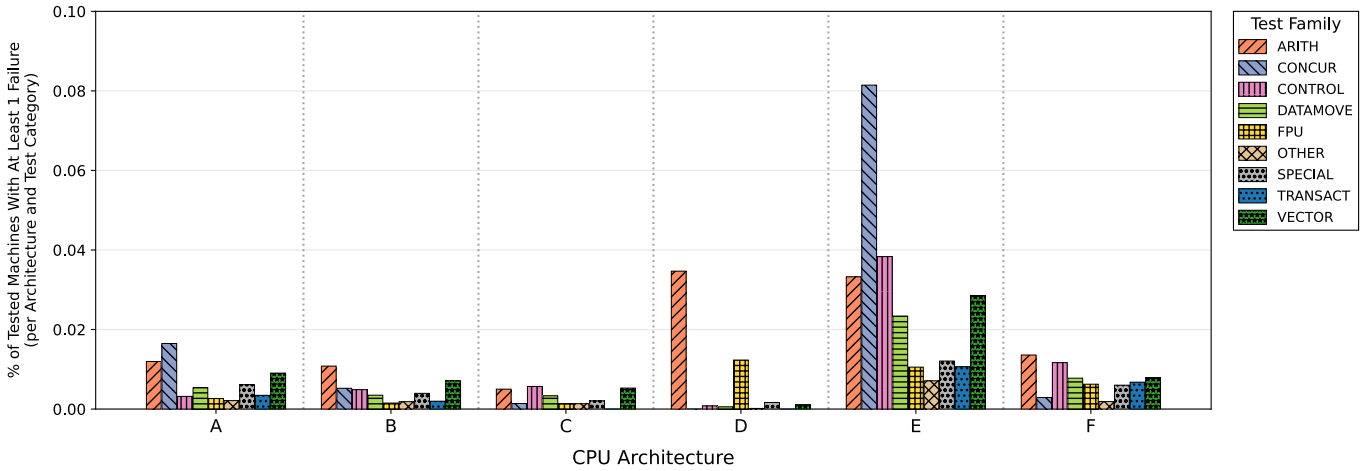

Fig. 7. Percentage proportion of all tested machines that experience at least one failure in each test family, split by architecture.

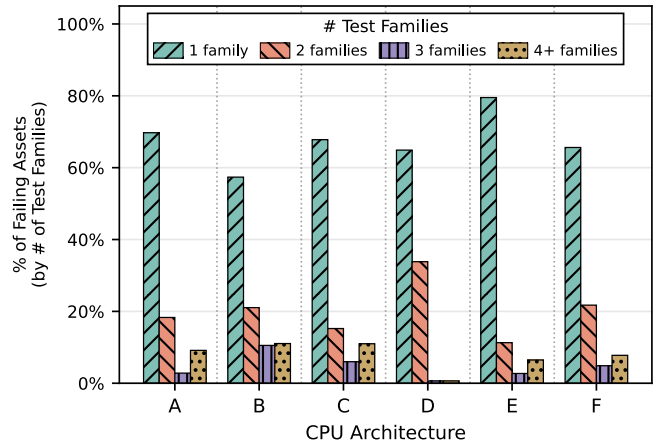

Fig. 8. Percentage proportion of failing machines experiencing 1/2/3/4+ test family failures, split by architecture.

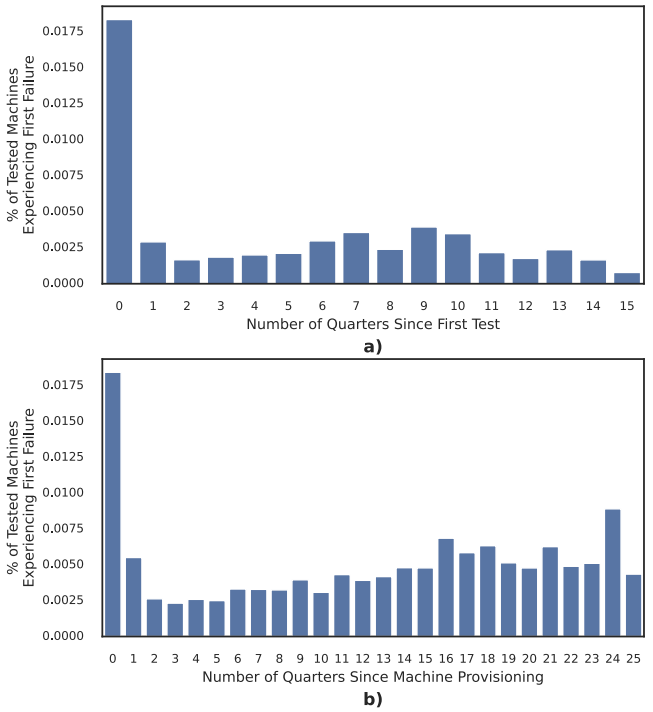

Fig. 9. Percentage of tested machines experiencing their first failure after some amount of time. a) denotes the percentage of machines that begin to fail in their nth quarter measuring from when testing (for a given test-suite) began. b) denotes the percentage of machines that begin to fail in their nth quarter measuring from the provisioning date of the machine.

Observation 5. An average of 0.0024% of tested machines begin failing in each quarter they are tested, not including the quarter in which testing began.

Notably, our observation of the average steady-state (i.e., after the initial quarter of testing) failure rate *per quarter* (0.0024%) is close to the *total* failure rate of all 'regular production' machines tested in [36] (0.0035%). This may suggest that our extended testing strategy was able to uncover significantly more failures over the lifespan of tested machines, compared to the 2-week test window employed by [36].

As some machines may have existed in the datacenter for years before any SDC tests were deployed, we additionally study how old machines were at first test-suite failure to better understand the potential impact of machine aging. Figure 9b) shows the proportion of tested machines that experience their first failure after being provisioned for some number of quarters. We again observe a clear indication of infant mortality in the first quarter that a machine has been provisioned. Past the initial burn-in period, we observe that machines still experience failures at significant (and relatively constant) rates throughout the rest of their lifespan. While this result may suggest that we are observing new wear-out failures, it may also be possible that a constantly-defective machine (with intermittent/low failure probability) evaded detection due to the stochastic nature of the tests. We do not observe any notable increase in failures amongst older machines (i.e., wearout-failures on the bathtub curve [17]), however our machine population may simply not be old enough for appreciable

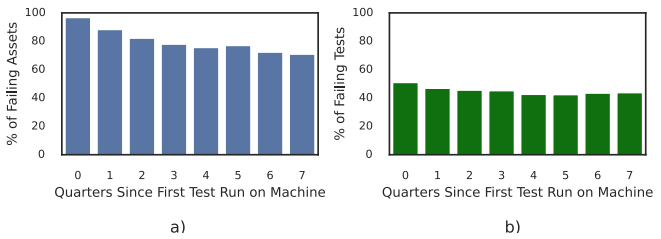

Fig. 10. Reproducibility of failures in machines undergoing prolonged testing loop reservation holds. Fig a) is the percentage of machines that continue to experience failures each quarter, Fig b) is the percentage of test runs that fail on these machines each quarter.

amounts of these failures to occur. While we observe a slight upward-trend in Figure 9b), this small increase may be attributed to older machines with architectures that have higher failure rates (as they make up a larger proportion of machines tested of that age).

Continued Failure Rate over Time. For machines which have long-term testing loop reservation holds (and have thus failed at least once, i.e., those suspected to be defective), we also measure the long-term behavior of these machines. In this analysis, we examine machines that have long-term testing loop reservation holds and have been tested at least once per quarter for at least 8 consecutive quarters (2 years), measured from their first failure.

Observation 6. Amongst machines which have been observed to fail, most ($> 71\%$) continue to fail at a steady rate when tested further.

Figure 10a) shows the percentage of considered machines that continue to fail in the testing reservation hold. We observe that a majority ($> 71\%$) of machines considered continue to fail for at least two years after continued testing. There are also instances where machines which were failing suddenly ceased to fail – one machine which began testing in August 2022 experienced 119 separate vector-instruction related test failures between June 2023 and March 2024, then ceased to experience a single additional failure to-date (after 59k+ test runs since the last observed failure), even despite replaying previously failing test seeds. This machine did not undergo any hardware, software, or firmware changes during this testing period, and was not relocated or exposed to different ambient conditions. Machines such as these illuminate the complex cases which can be revealed when testing for SDCs. In aggregate however, the machines considered tend to reliably continue to fail – Figure 10b) demonstrates that the overall failure rate of all tests performed stays roughly constant. We additionally do not observe any sensitivity to variations in test failure percentage across all test classes considered. These results further emphasize the need for rigorous SDC testing, as a majority of faulty machines will continue to fail in the field.

VI. Failure Characterization Deep-Dive

Provided an understanding of high-level failure behaviors across both architecture and time, we now zoom in to study detailed observations across a subset of tests performed in the fleet. In Section VI-A, we'll first examine detailed failure information from a broad-scope test-suite covering all failure modes described in Section IV, drawing insights into which particular tests and cores tend to fail most often. Making the observation that vector-related tests are a commonly observed failing class of tests, we then examine specific failures gathered from a narrower scoped vector test-suite.

A. Observations from a Broad-Scoped Test-Suite

We begin by examining a broad-scope test-suite including the same tests as those studied in [36]. Our test-suite has however regularly been updated to include new tests over time. Among the tests available in the test-suite, a random selection is made from those with the highest efficacy for the architecture under test, taking into account both the architecture itself and the time budget available. These selected tests are then executed while ensuring that the total test duration stays within the reserved time constraints. In our analyses, we exclude 5 tests in the broad-scoped test-suite that we have observed to exhibit SDC false-positives during the course of the study.

Observation 7. Across all tests in the broad-scoped test-suite 91.4% were observed to fail at least once, indicating that a vast majority of tests in this suite are well-suited to detect failures.

Previous work leveraging a limited set of tests within a similar suite [36] observed that only a small minority of tests ($< 15\%$) were able to identify any failures – our higher observed test coverage can likely be attributed to our significantly larger machine pool and extended testing time. Notably, in our results, we observe that 31% of individual tests (in the order of hundreds of tests) were observed to be the only failing test on at least one machine, further conveying the importance of a diverse test set.

Commonly Failing Tests. While the exact details of the tests in the broad-scoped suite are proprietary, we now share a brief commentary on the application behaviors associated with the most commonly failing tests. A high-level breakdown of the relative frequency of failures observed per test family and architecture is shown in Figure 11.

We generally observe that the most commonly failing application patterns involve vector matrix calculations with convolutional layers and dimensional reduction (commonly used in machine learning applications), closely followed by general purpose arithmetic operations used in linear algebraic operations. Other common failure modes involve 1) concurrency operations where threads are contending for the same resource however the resource release handshake encounters a data corruption, 2) non-locking-related concurrency operations

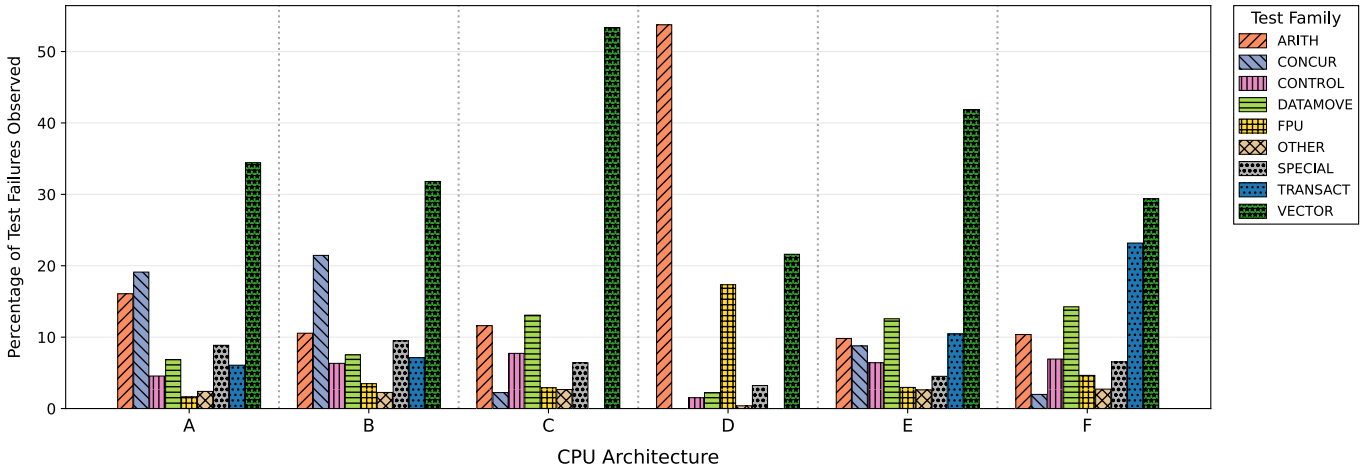

Fig. 11. Relative percentage breakdown of test failure rates across all test families in the broad-scoped test-suite, normalized by architecture.

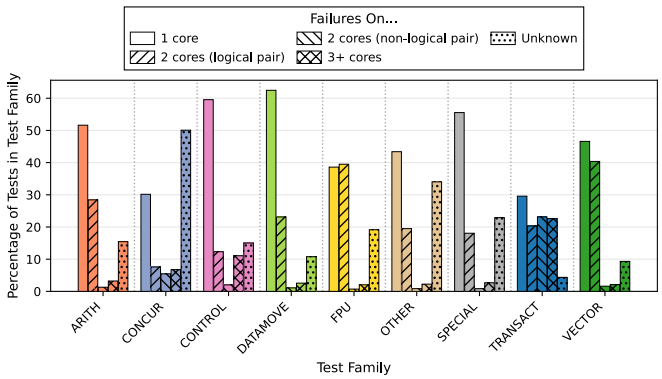

Fig. 12. Percentage proportion of failing tests experiencing 1/2/3+ simultaneous logical core failures, split by test family.

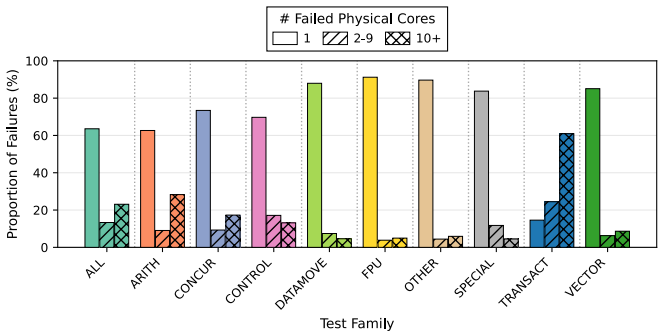

Fig. 13. Percentage proportion of machines which experience at least one failure on one, two to nine, and at least ten physical cores on that machine.

(such as transactional memories), 3) complex instruction flows, and 4) large-scale collective and data movement operations employed in distributed systems as well as artificial intelligence applications. Tests representing these application types are the most frequent failures in infrastructure. A corollary of this result is that almost all the commonly observed applications at hyperscale are affected by SDCs with high frequency.

CPU Core Failure Affinity. For most tests in the broad-scoped test-suite, information is gathered as to exactly which individual cores experienced a failure during testing. Leveraging this information, we first analyze how often multiple errors are experienced during the execution of one test, and how often these errors are co-located in the same physical core. For each test for which core information is available, we measure the proportion of failures where one, two, or more than two logical cores failed during the duration of the test. We further break down the two core case into two sub-categories, one where two sibling-cores fail (i.e., failures mapping to the same physical core), and when two non-related logical cores fail (in all cases examined, only two simultaneous threads are run on a single physical core). This core failure breakdown across different failure modes is shown in Figure 12.

Observation 8. Co-located threads on the same physical core regularly fail together, while simultaneous failures during a test on different physical cores are rare.

In general, most tests only experience one logical core failure at a time. This is to be expected even in the case where a defect exists in shared compute logic between two cores, as failures are often unpredictable and depend on intermittent hardware defect behaviors and/or exactly which inputs were processed. In the cases where two separate logical core failures are observed during the same test however, these failures are predominantly located within the same physical core. We note that when multiple logical cores fail during a test, this suggests *but does not imply* that the failure stems from a shared structure between those two logical cores – in some tests, it may be possible that a silently incorrect value from a defective logical core causes a software bug/side-effect in a non-defective logical core (e.g., a segmentation fault due to out-of-bounds access). We observe that tests that exercise transactional memory have a particular lack of core affinity, suggesting that these failures may be related to components shared across physical cores (such as shared memories).

Since an individual test run may not always detect all failing cores in a single run, the results shown in Figure 12 are not

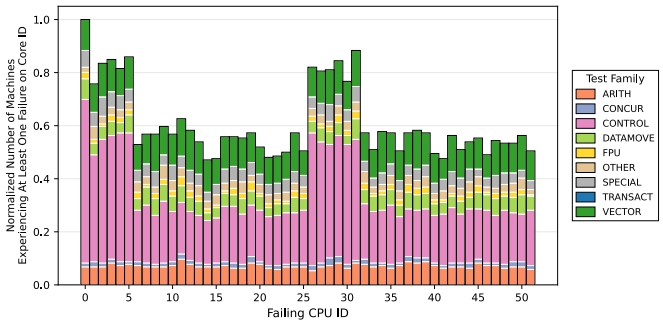

Fig. 14. Normalized number of machines experiencing at least one failure on each core ID for an example CPU with a significant cluster population.

indicative of potential defectivity across the entire chip. To better assess how many cores tend to exhibit failures across an entire chip, Figure 13 shows how often a machine has been observed to have more than one physical core failure across *all* tests ever run on that machine (which have core failure information). The data in Figure 13 considers both failures aggregated across all test families, in addition to only considering the core failure patterns of specific test families.

Observation 9. In more than 60% of cases, a machine will only experience SDC test failures on a single physical core, however multi-physical-core failures are not uncommon.

When considering failures across all test families, we observe approximately 62% of faulty machines experiencing failures on a single physical core, with 14% having 2-9 failing physical cores, and 24% having at least 10 failing physical cores. We note that these physical core observations do not align with those made in [36], who observed that half of CPUs exhibit failures on one physical core, while the other half exhibit failures on *all* physical cores. Our observations instead suggest that failures across a significant number of physical cores are actually rare (all CPUs examined had more than 10 cores) – this discrepancy may potentially be explained by the fact that several of the 5 false-positive tests we previously identified in the test-suite indicated failures across all cores. Interestingly, when considering only transactional memory tests, 10+ core failures are more common compared to less than 10, potentially further suggesting a shared root-cause across physical cores.

While the exact underlying hardware causes of SDC test failures is unknown (and outside the scope of this paper), we further look for hints as to how other details may influence the rate at which SDC tests fail. To do so, we further examine how failures occurring on a given CPU model are correlated with individual core IDs. Figure 14 examines one particular CPU model well represented in Meta's CPU fleet, plotting the normalized proportion of failing machines which experienced at least one failure on each core ID (again, split by test family).

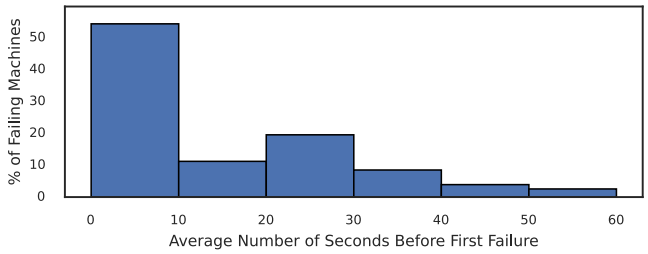

Fig. 15. Average number of seconds before first test failure is observed when running tests from vector-based test-suite.

Observation 10. While uncommon, a subset of tests exhibit a tendency to experience failures on certain core IDs more than others.

Across all test families assessed in Figure 14, we generally observe similar numbers of machines experiencing failures across each core ID considered. One notable exception visible in Figure 14 can be seen – elevated failures occur in cores 0-5 (and sibling cores 26-31). These elevated failures are attributed to control-family SDCs detected when testing features used to identify the capabilities of the processor across cores. Due to limited debugging information, we do not know the underlying root-cause of these elevated failures – while they may be related due to the cores being physically adjacent, it also may be the case that these identifier features have issues on this particular core when computing small core IDs. We further observe similar skewed behaviors on some CPUs of different architectures wherein a test is observed to cause core 0 related failures on 3x as many machines compared to any other core ID.

B. Observations from Vector Test Family

While the results provided by the broad-based test-suite present an excellent comprehensive view of the failure landscape, the breadth associated with the suite limits the ability to log operational and instructional details associated with failures. Given the high number of vector-related failures observed (as highlighted in Figure 11), we now zoom further into studying a set of focused application-representative tests specifically representing large-scale vector computations. These tests encompass 450+ vector instruction behaviors, additionally exercising these instructions across different vector sizes (culminating in over two-thousand different test variants in total) across randomized input spaces. On each machine tested, the vector application-style tests execute an average of over 9 billion individual sequences per execution, and the sensitivity analysis for test duration to detection efficacy is captured in Figure 15.

We observe that high-throughput, hyper-specific vector application-style tests are often able to catch errors quickly. As seen in Figure 15, for a majority of failing machines, the average detection latency is less than 10 seconds, highlighting that targeted application-style tests' effectiveness in the time-constrained testing environment.

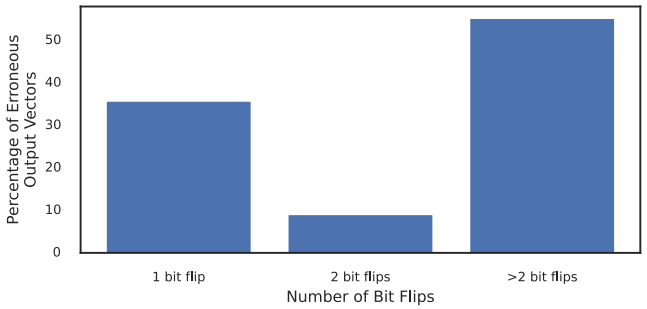

Fig. 16. Number of bit-flips observed across vector instruction outputs when considering vector-focused test-suite.

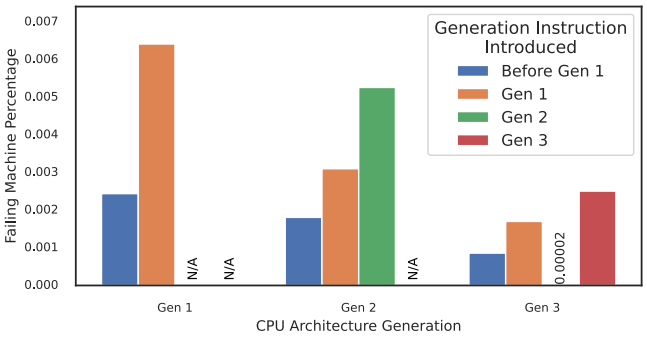

Fig. 17. Percentage of machines which fail vector-focused tests, broken down across the architectural generation of the machine and the architectural generation when the tested vector instruction was introduced. Gen 2 instructions run on Gen 3 were observed to have a negligible (but non-zero) number of failures.

Common Failure Patterns. Across all instructions tested, we observe that single and double-precision vector fused multiply (vfm) instructions have the highest failure rates of all instructions studied. These vfm instructions perform a complex chain of vector operations (e.g., multiplying two packed floating-point vectors, adding another packed floating-point vector, then rounding/storing the result). These sequences are also the most commonly used set of calculation patterns in large-scale machine learning and artificial intelligence applications, among all the vector instruction sequences. Other frequent offenders that we observe include dot-products, square-root, and other arithmetic operations (e.g. add/sub/mul/div) on packed floating-point vectors. We observe that as we further move into larger models for training and inference, as well as GenAI applications, which predominantly leverage large-scale floating-point-based General Matrix Multiply (GEMM) and Dot Product Engines (DPE), the rate of corruption across these applications increases. Given the increasing importance of vectorized applications, a deeper study into the nuances of each instruction, including dependencies on width and heterogeneity of vector instruction sequences, is warranted and a topic we intend to elaborate on in a subsequent publication.

We also examine the impact that detected SDCs have on the output of a faulty instruction. The average number of bit-flips across instructions' output vectors (gathered across all failed test runs over a two-week window) is shown in Figure 16.

> **Observation 11.** Multi bit-flips are observed to outnumber single bit-flips in vector instruction outputs.

This result impresses the importance to consider diverse fault-modes separate from random bit-flips, corroborating the finding from [36] that single bit-flip models are often not sufficient to reason about SDCs. We note that this observation suggests a significantly higher proportion of multi-bit-flips compared to that reported in [36], even when examining the number of bit-flips in individual vector elements (rather than the entire output vector). Notably, in approximately 90% of cases observed, only a single output vector element experiences bit-flips, suggesting that in most (but not all) cases, errors only affect a single element of the vector. We find no biases towards which element in a vector experiences a fault.

Failure Rates over CPU Generations. In addition to studying individual instruction failures, we also collect aggregate statistics on how different instruction classes behave across different CPU generations. In Figure 17, we measure the percentage of machines which experience failures in vector tests across three CPU architecture generations for which we have a statistically-significant test sample. We further categorize the tests based on when which CPU generation the tested functionality was introduced.

> **Observation 12.** Instructions tend to fail at higher rates in the architectural generation that they are introduced, with subsequent generations having lower failure rates for those features.

In each successive generation, we observe a positive drive in reducing the particular failures observed in the previous generations. These results further demonstrate the importance of continued development of tests, as new instructions to a particular generation can expose new risks.

VII. Related Work

We refer to Section II-C for a comprehensive summary of at-scale SDC testing works.

SDC test generation strategies employed in industry have been described by Google [27], Intel [21], [30], and Tesla [33]. Harpocrates [16] offers a functional test generation approach that aims to maximize SDC test efficacy via microarchitectural simulation.

Theorized SDC behaviors have been studied under a variety of assumed failure modes, including particle-strikes [3], [10], [24], stuck-at-bits [21], and delay faults [14], [29], [32]. Further academic efforts have been made to assess program and system-level vulnerability to SDCs via circuit, architectural, and software level modeling [1], [4], [5], [10], [13], [19], [25], [26]. While these works provide detailed insights into

how failures may propagate through systems, they are limited to studying significantly simpler cores and architectures compared to those present in a production fleet.

VIII. Conclusion

This work presents PinDrop, the most comprehensive characterization of Silent Data Corruptions (SDCs) in production systems to date, leveraging over 500 million test executions across millions of servers to reveal crucial insights into SDC behavior at hyperscale. For the first time, PinDrop's continuous testing methodology enables the assessment of failure trends over time, identifying an order of magnitude more defective parts than prior work. Our findings demonstrate that SDCs are very prevalent, with new failures emerging even after years of continuous testing. We share and categorize numerous insights related to test families within the failure state space. Failures across each test and instruction family studied, along with their associated architectural dependencies captured in the paper, can benefit from further research. These results underscore the significant importance of continuous, high-frequency testing infrastructure rather than snapshot-based approaches, while revealing that SDCs affect a broad spectrum of computational features.

IX. Acknowledgements

The authors extend special gratitude to Aslan Bakirov, Melita Mihaljevic, Laura Boyle, and Rik Van Riel for their enduring partnership in enabling production-facing detection methodologies. We thank Vijay Rao, Manish Modi, Chunqiang (CQ) Tang, Daniel Moore, Philip Harries, Amitav Mohanty, Steve Kehoe, Jacqueline Jiang, and numerous other infrastructure engineers whose support and contributions were instrumental to the methodologies presented in this paper. Additionally, we acknowledge Richa Mishra, Anuj Mittal, and Ranjith Meka for their efforts in managing analytical data pipelines. We are grateful to the anonymous reviewers for their valuable feedback.

References

[1] U. Agarwal, A. Chan, and K. Pattabiraman, "Resilience assessment of large language models under transient hardware faults," in *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. Los Alamitos, CA, USA: IEEE Computer Society, oct 2023, pp. 659–670. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/ISSRE59848.2023.00052

[2] L. N. Bairavasundaram, A. C. Arpaci-Dusseau, R. H. Arpaci-Dusseau, G. R. Goodson, and B. Schroeder, "An analysis of data corruption in the storage stack," *ACM Trans. Storage*, vol. 4, no. 3, nov 2008. [Online]. Available: https://doi.org/10.1145/1416944.1416947

[3] R. Baumann, "Radiation-induced soft errors in advanced semiconductor technologies," *IEEE Transactions on Device and Materials Reliability*, vol. 5, no. 3, pp. 305–316, 2005.

[4] O. Chatzopoulos, N. Karystinos, G. Papadimitriou, D. Gizopoulos, H. D. Dixit, and S. Sankar, "Veritas – demystifying silent data corruptions: uarch-level modeling and fleet data of modern x86 cpus," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1–14.

[5] P. W. Deutsch, V. Q. Ulitzsch, S. Gurumurthi, V. Sridharan, J. S. Emer, and M. Yan, "Delayavf: Calculating architectural vulnerability factors for delay faults," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024, pp. 231–245.

[6] H. D. Dixit, L. Boyle, G. Vunnam, S. Pendharkar, M. Beadon, and S. Sankar, "Detecting silent data corruptions in the wild," *arXiv preprint arXiv:2203.08989*, 2022.

[7] H. D. Dixit, S. Pendharkar, M. Beadon, C. Mason, T. Chakravarthy, B. Muthiah, and S. Sankar, "Silent data corruptions at scale," 2021. [Online]. Available: https://arxiv.org/abs/2102.11245

[8] S. Dong, A. Kryczka, Y. Jin, and M. Stumm, "Rocksdb: Evolution of development priorities in a key-value store serving large-scale applications," *ACM Trans. Storage*, vol. 17, no. 4, oct 2021. [Online]. Available: https://doi.org/10.1145/3483840

[9] R. Dutta, H. D. Dixit, R. Van Riel, G. Vunnam, and S. Sankar, "Hardware sentinel: Protecting software applications from hardware silent data corruptions," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025, pp. 482–497.

[10] S. K. S. Hari, P. Rech, T. Tsai, M. Stephenson, A. Zulfiqar, M. Sullivan, P. Shirvani, P. Racunas, J. Emer, and S. W. Keckler, "Estimating silent data corruption rates using a two-level model," *arXiv preprint arXiv:2005.01445*, 2020.

[11] Y. He, M. Hutton, S. Chan, R. De Gruijl, R. Govindaraju, N. Patil, and Y. Li, "Understanding and mitigating hardware failures in deep learning training systems," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3579371.3589105

[12] P. H. Hochschild, P. J. Turner, J. C. Mogul, R. K. Govindaraju, P. Ranganathan, D. E. Culler, and A. Vahdat, "Cores that don't count," in *Proc. 18th Workshop on Hot Topics in Operating Systems (HotOS 2021)*, 2021.

[13] Y. Huang, S. Guo, S. Di, G. Li, and F. Cappello, "Mitigating silent data corruptions in hpc applications across multiple program inputs," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022, pp. 1–14.

[14] H. Jafarzadeh, F. Klemme, J. D. Reimer, Z. P. Najafi-Haghi, H. Amrouch, S. Hellebrand, and H.-J. Wunderlich, "Robust pattern generation for small delay faults under process variations," in *2023 IEEE International Test Conference (ITC)*, 2023, pp. 111–116.

[15] X. Jiao, A. Pandey, K. Pattabiraman, and F. Lin, "Large-scale ai infra reliability: Challenges, strategies, and llama 3 training experience," in *2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks - Supplemental Volume (DSN-S)*, 2025, pp. 140–146.

[16] N. Karystinos, O. Chatzopoulos, G.-M. Fragkoulis, G. Papadimitriou, D. Gizopoulos, and S. Gurumurthi, "Harpocrates: Breaking the silence of cpu faults through hardware-in-the-loop program generation," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 516–531.

[17] G.-A. Klutke, P. C. Kiessler, and M. A. Wortman, "A critical look at the bathtub curve," *IEEE Transactions on reliability*, vol. 52, no. 1, pp. 125–129, 2003.

[18] G. Lee, B.-G. Chun, and H. Katz, "Heterogeneity-aware resource allocation and scheduling in the cloud," in *Proceedings of the 3rd USENIX Conference on Hot Topics in Cloud Computing*, ser. HotCloud'11. USA: USENIX Association, 2011, p. 4.

[19] Z. Li, H. Menon, D. Maljovec, Y. Livnat, S. Liu, K. Mohror, P.-T. Bremer, and V. Pascucci, "Spotsdc: Revealing the silent data corruption propagation in high-performance computing systems," *IEEE Transactions on Visualization and Computer Graphics*, vol. 27, no. 10, pp. 3938–3952, 2021.

[20] J. Ma, H. Pei, L. Lausen, and G. Karypis, "Understanding silent data corruption in llm training," *arXiv preprint arXiv:2502.12340*, 2025.

[21] T. Macieira, S. Gurumurthy, S. Gurumurthi, A. Haggag, G. Papadimitriou, and D. Gizopoulos, "Silent data corruptions in computing: Understand and quantify," in *2024 IEEE 30th International Symposium on On-Line Testing and Robust System Design (IOLTS)*. IEEE, 2024, pp. 1–7.

[22] S. E. Michalak, K. W. Harris, N. W. Hengartner, B. E. Takala, and S. A. Wender, "Predicting the number of fatal soft errors in los alamos national laboratory's asc q supercomputer," *IEEE Transactions on Device and Materials Reliability*, vol. 5, no. 3, pp. 329–335, 2005.

[23] S. Mitra, B. Parthasarathy, S. Banerjee, R. Govindaraju, P. Hochschild, E. Liu, M. Fuller, M. Dixon, and P. Ranganathan, "Silent data corruption by 10× test escapes threatens reliable computing," *IEEE*

Design & Test, vol. 42, Issue 6, pp. 40–53, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/11142270

[24] S. Mukherjee, J. Emer, and S. Reinhardt, "The soft error problem: an architectural perspective," in *11th International Symposium on High-Performance Computer Architecture*, 2005, pp. 243–247.

[25] G. Papadimitriou and D. Gizopoulos, "Silent data corruptions: Microarchitectural perspectives," *IEEE Transactions on Computers*, vol. 72, no. 11, pp. 3072–3085, 2023.

[26] G. Papadimitriou, D. Gizopoulos, H. D. Dixit, and S. Sankar, "Silent data corruptions: The stealthy saboteurs of digital integrity," in *2023 IEEE 29th International Symposium on On-Line Testing and Robust System Design (IOLTS)*, 2023, pp. 1–7.

[27] K. Serebryany, M. Lifantsev, K. Shtoyk, D. Kwan, and P. Hochschild, "Silifuzz: Fuzzing cpus by proxy," *arXiv preprint arXiv:2110.11519*, 2021.

[28] R. Sethi, M. Traverso, D. Sundstrom, D. Phillips, W. Xie, Y. Sun, N. Yegitbasi, H. Jin, E. Hwang, N. Shingte *et al.*, "Presto: Sql on everything," in *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. IEEE, 2019, pp. 1802–1813.

[29] M. Shamsa and D. Lerner, "Defect mechanisms responsible for silent data errors," in *2024 IEEE International Reliability Physics Symposium (IRPS)*, 2024, pp. 1–5.

[30] M. Shamsa, J. D. Martin, M. Phielipp, T. Macieira, L. Lingappan, B. Kelly, D. Lerner, M. Tucknott, and E. Hansen, "Improved silent data error detection through test optimization using reinforcement learning," in *2025 IEEE International Reliability Physics Symposium (IRPS)*. IEEE, 2025, pp. 8C–1.

[31] A. Singh, S. Chakravarty, G. Papadimitriou, and D. Gizopoulos, "Silent data errors: Sources, detection, and modeling," in *2023 IEEE 41st VLSI Test Symposium (VTS)*, 2023, pp. 1–12.

[32] G. Sudhanva, S. Vilas, and G. Sankar, "Emerging fault modes: Challenges and research opportunities," 2023. [Online]. Available: https://www.sigarch.org/emerging-fault-modes-challenges-and-research-opportunities/

[33] Tesla AI, "Functional simulator for tesla dojo," 2025. [Online]. Available: https://x.com/Tesla_AI/status/1939640913661206733

[34] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at google with borg," in *Proceedings of the tenth european conference on computer systems*, 2015, pp. 1–17.

[35] S. Vigraham and B. Leonhardi, "Maintaining large-scale ai capacity at meta," 2024. [Online]. Available: https://engineering.fb.com/2024/06/12/production-engineering/maintaining-large-scale-ai-capacity-meta/

[36] S. Wang, G. Zhang, J. Wei, Y. Wang, J. Wu, and Q. Luo, "Understanding silent data corruptions in a large production cpu population," in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 216–230. [Online]. Available: https://doi.org/10.1145/3600006.3613149